

the message is to be sent because no prior connection is established in UDP communication.

```
int sendto ( int sid, const void *bufferPtr, size_t
bufferLength, int flag,
struct sockaddr *addrPtr, socklen_t addrLength );
```

Here *addrPtr* specifies the address of the node to which data is sent. It returns the number of bytes sent or -1 in case of failure.

recvfrom: The *recvfrom* API is the UDP counterpart of the TCP API *recv*. *recvfrom* obtains the sender address as additional information from the variable *addrPtr*.

```
int recvfrom ( int sid, void *bufferPtr, int bufferLength,
int flag, sockaddr *addrPtr, int *addrLengthPtr );
```

The *recvfrom* API returns the number of bytes received or -1 in case of failure.

Socket programming header files

The socket programming APIs are not provided by a single header file. The header files `<sys/types.h>` and `<sys/socket.h>` are included in every program to access functions like *socket*, *bind*, *listen*, *accept*, *connect*, *send*, *recv*, *sendto*, *recvfrom*, etc. The header file `<unistd.h>` part of the POSIX library is included to access the *close* function. The header file `<netinet/in.h>` is included to use structures like *sockaddr_in*, which is used to store addresses for the Internet address family. To access the *gethostbyname* function, which searches the database and finds an entry that matches the host name specified by the name argument, we use the header file `<netdb.h>`. The header file `<arpa/inet.h>` is included to use the function *htons*, which converts an unsigned short integer from host byte order to network byte order. There are many other header files and functions in socket programming. But I am only discussing the bare minimum, which are absolutely essential.

An example of TCP client-server communication

Now let us look at an example where a TCP connection is established between a client and a server:

```
// TCP server program tcp_server.cc
#include<iostream>
#include<cstdlib>
#include<cstring>
#include<unistd.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>

using namespace std;
```

```
int main() {
    int sockfd, newsockfd, portno;
    socklen_t cli;
    char buffer[256];
    struct sockaddr_in serv_addr, cli_addr;
    int n;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    bzero((char *) &serv_addr, sizeof(serv_addr));
    portno = 3333;
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = INADDR_ANY;
    serv_addr.sin_port = htons(portno);
    bind(sockfd, (struct sockaddr *) &serv_addr,
sizeof(serv_addr));
    listen(sockfd,1);
    cli = sizeof(cli_addr);
    newsockfd = accept(sockfd, (struct sockaddr *) &cli_
addr, &cli);
    bzero(buffer,256);
    n = read(newsockfd,buffer,255);
    cout<<"Message Received from Client : "<<buffer<<"\n\n";
    n=send(newsockfd,"Your Message Received",21,0);
    close(newsockfd);
    close(sockfd);
    return 0;
}
```

Both the client and server programs do not contain any code for error checking in order to reduce the size. The function *bzero* copies zeros on an array. The port number assigned to the server is 3333. The server listens to this port for the incoming 'connect' request from the client. The client should send its requests to this port checked by the server for connections. The port number should be that of an ephemeral port ranging between 1024 and 49151. BSD sockets often use port numbers between 1024 and 5000. Figure 2 shows the terminal executing the server program.

```
// TCP client program tcp_client.cc
#include<iostream>
#include<cstdlib>
#include<unistd.h>
#include<cstring>
#include<stdio>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<netdb.h>

using namespace std;

int main() {
    int sockfd, portno, n;
    struct sockaddr_in serv_addr;
```